

# UrbanMind 2.0

Autonomous Multi-Agent Traffic Intelligence System  
for Bangalore

## Technical Documentation

Repository: [github.com/haasa-x/urbanmind](https://github.com/haasa-x/urbanmind)  
Version 2.0 · Compiled September 04, 2026  
Author: haasa-x

# Table of Contents

|                                                    |    |
|----------------------------------------------------|----|
| 1. Executive Summary .....                         | 3  |
| 2. System Architecture .....                       | 4  |
| 3. Technology Stack .....                          | 6  |
| 4. Repository Layout .....                         | 8  |
| 5. Backend Deep-Dive .....                         | 10 |
| 5.1 main.py — FastAPI Entry Point .....            | 10 |
| 5.2 The Agent Layer .....                          | 13 |
| 5.3 The SupervisingAgent .....                     | 16 |
| 5.4 Utilities and Services .....                   | 17 |
| 5.5 The src/ subsystem: RAG, Routing, Vision ..... | 18 |
| 6. Frontend Deep-Dive .....                        | 21 |
| 7. Data Flow (Citizen → Agents → Dashboards) ..... | 24 |
| 8. API Reference .....                             | 26 |
| 9. LLM + RAG Pipeline .....                        | 28 |
| 10. Networking: SSE, HTTP, CORS, Broadcast .....   | 30 |
| 11. System Design Concepts .....                   | 32 |
| 12. Logging Subsystem .....                        | 35 |
| 13. Deployment and Operations .....                | 37 |
| 14. Future Work .....                              | 39 |
| 15. Appendix: Command Reference .....              | 40 |

# 1. Executive Summary

UrbanMind 2.0 is an autonomous, multi-agent traffic intelligence platform designed to observe, reason about, and coordinate a response to road-network incidents in Bangalore, India. It combines Large-Language-Model-driven agents, a keyword-scored Retrieval-Augmented-Generation (RAG) knowledge layer, computer vision for evidence verification, and real road-network routing into a single, operator-facing dashboard supported by dedicated portals for each responding department (Traffic Police, Hospital, Fire, Public Information, BBMP, and Ambulance Crew). The system also exposes a citizen-facing page for reporting incidents and looking up routes.

The system is built as a full-stack web application. The backend is a single asynchronous FastAPI process that hosts twelve specialized agents whose execution order is decided per-incident by a SupervisingAgent. Reasoning is delegated to Groq's **llama-3.3-70b-versatile** model. Image evidence is analyzed on-device using Apple's **mobilevit-small** vision transformer. Real-world driving routes are computed via the public OSRM API, geocoding via Nominatim, and city geometry displayed with Leaflet.js on top of Stadia Maps' Alidade Smooth Dark tiles.

State is broadcast to every connected client through Server-Sent Events (SSE) using a per-subscriber fan-out queue. The React 18 frontend uses Vite for its development server and React Router for the six standalone department portals. The whole application runs locally — no database is required; state is held in a single in-memory **CityState** singleton with bounded lists for safety.

## Key capabilities

- Ten LLM-powered agents plus a supervisor and a comms officer, wired through a lightweight LangGraph-compatible orchestrator.
- Severity-driven behaviour matrix (levels 1–5) that scales unit dispatch, corridor rules, agent participation and metrics multipliers.
- Capability-first hospital selection that unifies EMS dispatch requirements with paramedic field observations.
- MobileViT-backed image evidence classifier that auto-escalates severity when injury indicators are visually confirmed.
- OSRM road-following ambulance corridor rendered on-map for the operator and ambulance crew dashboards.
- Structured, colour-tagged logging pipeline so every agent decision is observable in the terminal in real time.
- SSE broadcast fan-out that survives back-end restarts thanks to a 2-second auto-reconnecting client.

## 2. System Architecture

UrbanMind adopts a hub-and-spoke architecture. A single FastAPI process is the hub. Every dashboard — operator, citizen, ambulance, and each department portal — is a spoke that opens its own SSE connection and listens to the same event stream. When any incident is reported, the hub runs an agent orchestration graph and pushes each agent’s decisions to every spoke simultaneously, so all views stay in lock-step without needing any peer-to-peer coordination between browser tabs.

### Runtime topology

| Component         | Process        | Port | Purpose                                        |
|-------------------|----------------|------|------------------------------------------------|
| FastAPI (main.py) | uvicorn worker | 8000 | REST + SSE hub, agent orchestration, LLM calls |
| Vite dev server   | node           | 5173 | Serves the React SPA + HMR                     |
| Groq API          | cloud          | 443  | LLM inference (llama-3.3-70b-versatile)        |
| OSRM public API   | cloud          | 443  | Real road routing                              |
| Nominatim         | cloud          | 443  | Reverse/forward geocoding                      |
| Stadia Maps       | cloud          | 443  | Dark map tiles                                 |
| Hugging Face      | cloud          | 443  | First-run MobileViT weight download            |

### Logical layers

The backend is organised into four logical layers, each with a single responsibility. The frontend mirrors that separation with an analogous split between the SSE consumer, the map component, agent-facing panels, and department portals.

| Layer         | Files                                        | Responsibility                                                   |
|---------------|----------------------------------------------|------------------------------------------------------------------|
| Transport     | main.py                                      | HTTP endpoints, SSE stream, request middleware, exception s      |
| Orchestration | agents/graph.py, agents/supervisor.py        | Agent sequencing, protocol selection, escalation, per-invocation |
| Agents        | agents/*.py                                  | Domain reasoning — one module per agent role. Each returns       |
| Services      | utils/*.py, src/vision, src/routing, src/rag | State, severity config, alert dispatcher, keyword RAG, OSRM/N    |

### Event-driven fan-out

The core communication pattern is publish/subscribe over Server-Sent Events. Every incoming HTTP request that produces state changes emits typed events into an internal broadcast function, which pushes them into the tail of every connected subscriber’s queue. Because every subscriber owns its own `asyncio.Queue`, no event is ever consumed by only one browser tab — the previous shared-queue design was the primary source of “agents not showing up on the operator dashboard” bugs and has been replaced entirely.

## 3. Technology Stack

### Backend

| Library / Service | Version | Purpose                                                         |
|-------------------|---------|-----------------------------------------------------------------|
| Python            | 3.10+   | Runtime for the whole backend.                                  |
| FastAPI           | latest  | Async HTTP server, dependency injection, routing.               |
| uvicorn           | latest  | ASGI worker that hosts FastAPI.                                 |
| httpx             | latest  | Async HTTP client — used for OSRM and Nominatim calls.          |
| python-dotenv     | latest  | Loads GROQ_API_KEY from backend/.env at startup.                |
| python-multipart  | latest  | Multipart form parsing (needed for /report image uploads).      |
| langchain-groq    | latest  | Thin wrapper around Groq's Chat Completions API.                |
| Groq              | SaaS    | Serves llama-3.3-70b-versatile inference.                       |
| transformers      | latest  | Loads and runs the MobileViT image classifier.                  |
| torch             | latest  | MobileViT backend.                                              |
| pillow            | latest  | Image decoding before feeding into MobileViT.                   |
| reportlab         | latest  | Optional PDF export of incident reports (this document itself). |

### Frontend

| Library / Service  | Version      | Purpose                                          |
|--------------------|--------------|--------------------------------------------------|
| React              | 18           | SPA rendering.                                   |
| Vite               | 5            | Dev server + HMR + production bundler.           |
| React Router DOM   | 6            | Client-side routing for /citizen, /dept/* pages. |
| Leaflet            | 1.x          | Map rendering.                                   |
| react-leaflet      | 4            | React bindings for Leaflet.                      |
| Stadia Maps        | SaaS         | Alidade Smooth Dark map tiles.                   |
| OSRM public server | SaaS         | Real driving routes.                             |
| Nominatim          | SaaS         | Free-form location geocoding.                    |
| Inter              | Google Fonts | Body typeface.                                   |
| JetBrains Mono     | Google Fonts | Numeric / timestamp / trace-log typeface.        |

### Development tools

- Node.js 18+ and npm for the frontend toolchain.
- Python venv for isolated backend dependencies.

- Git for version control, hosted on GitHub at [github.com/haasa-x/urbanmind](https://github.com/haasa-x/urbanmind).
- A supervisor shell script (`run.sh`) that clears stale processes on `:8000` and `:5173`, purges Vite's dep cache, launches both servers, and auto-restarts either one on crash.

## 4. Repository Layout

The commit history of the production build lays out as follows. Only paths that ship (no venv, node\_modules, .env, or model caches) are shown.

```
urbanmind/
■■■■ README-style docs generated separately
■■■■ run.sh                                - supervisor: kills stale procs, starts both servers
■
■■■■ backend/
■   ■■■■ main.py                          - FastAPI app, routes, SSE, logging, safety nets
■   ■■■■ replay.py                       - CLI that replays scripted scenario JSONs
■   ■■■■ requirements.txt
■   ■■■■ agents/
■     ■■■■ _llm.py                       - Groq client + graceful fallback
■     ■■■■ graph.py                     - Sequential orchestrator w/ SupervisingAgent gating
■     ■■■■ supervisor.py                 - Picks 1 of 5 protocols from an allow-list
■     ■■■■ incident.py                  - Classifies severity, type, spillover
■     ■■■■ prediction.py                 - T+5/15/30 cascade forecast
■     ■■■■ constraint.py                 - Validates against R-14B / R-07A / R-22C / R-31D / R-05E
■     ■■■■ intervention.py               - Ranks response options
■     ■■■■ emission.py                  - CO2 math
■     ■■■■ safety.py                    - POI-aware safety score
■     ■■■■ police.py                    - Unit dispatch, deployment instructions
■     ■■■■ ems.py                       - Mode 1 dispatch + Mode 2 field-assessment summariser
■     ■■■■ hospital_routing.py           - Capability-first hospital selection
■     ■■■■ emergency_corridor.py         - Green-wave corridor planning
■     ■■■■ protocol_dispatch.py          - Fans out department alerts
■     ■■■■ narrator.py                  - CommsAgent: operator, public, audit briefs
■   ■■■■ utils/
■     ■■■■ city_state.py                 - In-memory singleton with bounded lists
■     ■■■■ severity_config.py            - 5-level behavioural matrix
■     ■■■■ alert_dispatcher.py           - Templated department alerts
■     ■■■■ causal_graph.py               - Junction propagation weights
■     ■■■■ counterfactual.py             - What-if comparison model
■   ■■■■ src/
■     ■■■■ rag/
■       ■■■■ retriever.py                - Keyword-scored top-k over policy docs
■       ■■■■ knowledge/*.txt             - SOP corpus
■       ■■■■ routing/
■         ■■■■ osrm_router.py             - OSRM + Nominatim client, PRE_CACHED_ROUTES
■         ■■■■ vision/
■           ■■■■ analyzer.py              - MobileViT image analyzer
■   ■■■■ data/hospitals.json              - 30-hospital Bangalore roster w/ capabilities
■   ■■■■ scenarios/*.json                 - SC-01/02/03 scripted playback scripts
■
■■■■ frontend/
■■■■ package.json vite.config.js index.html
■■■■ src/
■■■■ main.jsx                            - React entry, BrowserRouter, Leaflet icon fix
■■■■ App.jsx                             - Dashboard, SSE consumer, route registry
■■■■ index.css                           - Design tokens + palette
■■■■ components/
■■■■                                     - Header, CityMap, AgentWorkflowPanel, AgentTraceLog,
■■■■                                     NarratorPanel, DepartmentAlertFeed, MetricsStrip,
■■■■                                     CockpitSidebar, CitizenView, CausalGraphSVG,
■■■■                                     TemporalMind, CounterfactualPanel, RoutePanel,
■■■■                                     ErrorBoundary
■■■■ components/departments/             - Police, Hospital, Fire, Public, BBMP, Ambulance,
```

|                            |                                                        |
|----------------------------|--------------------------------------------------------|
| ■                          | DeptLayout (shared wrapper)                            |
| ■■■ data/                  | – scenarios.js, severity.js, static HOSPITALS fallback |
| ■■■ hooks/useTypewriter.js | – Sequential typewriter with cancellable run-tokens    |
| ■■■ utils/densityColor.js  |                                                        |



## 5. Backend Deep-Dive

The backend is a single Python process. Everything below the FastAPI object hangs off in-memory state and is not persisted between restarts. Every external I/O — the Groq LLM, OSRM, Nominatim — is done asynchronously so the server can hold thousands of open SSE connections without blocking on any one of them.

### 5.1 main.py — FastAPI Entry Point

**main.py** is the transport and orchestration layer. It bootstraps FastAPI, registers routes, holds the shared CityState singleton, wires the SSE subscriber list, and hosts two safety nets that prevent any single bad request or async task from taking down the process. The important building blocks:

#### Global state

```
city_state = CityState()          # single in-memory truth
_subscribers: list[asyncio.Queue] = []  # fan-out – one queue per open /stream
_main_loop: asyncio.AbstractEventLoop = None
_CUSTOM_SEM = asyncio.Semaphore(3)    # caps concurrent /custom-incident runs
```

#### Never-crash HTTP middleware

Every request goes through **\_never\_500\_middleware**. If the route handler throws, the exception is caught, logged as [MW-CATCH], and a well-formed 500 JSON is returned. This is what stops the whole process from dying when, e.g., a Groq call transiently 429s.

```
@app.middleware("http")
async def _never_500_middleware(request, call_next):
    try:
        return await call_next(request)
    except Exception as e:
        print(f"[MW-CATCH] {request.method} {request.url.path} - {type(e).__name__}: {e}", flush=True)
        return JSONResponse(status_code=500, content={"error": f"{type(e).__name__}: {e}"})
```

#### Asyncio safety net

**\_report\_unhandled** is registered on the running event loop via `loop.set_exception_handler(...)`. If any background task (spawned by `asyncio.create_task`) raises without being awaited, the exception is printed rather than swallowed silently. Every explicit `create_task` call additionally attaches **\_task\_watchdog** as a done-callback for the same reason.

#### Fan-out broadcaster

The single most important piece of plumbing in the entire backend. Every event to be pushed to clients goes through **\_broadcast\_now**, which iterates the subscriber list and calls `put_nowait` on each. For thread-safety when the agent graph runs on `asyncio.to_thread`, **\_enqueue\_sync** wraps the same call in `loop.call_soon_threadsafe(...)` so the writes happen on the event loop thread and never race the readers.

```
def _broadcast_now(evt: dict):
    for q in list(_subscribers):
        try: q.put_nowait(evt)
        except Exception: pass

def _enqueue_sync(evt: dict):
```

```

if _main_loop is None:
    _broadcast_now(evt); return
_main_loop.call_soon_threadsafe(_broadcast_now, evt)

```

■ *Historical bug: a single shared `asyncio.Queue` meant every event was consumed by exactly one browser tab. Switching to per-subscriber queues was the fix that made multi-tab operator + citizen + department portals all reflect the same incident at the same time.*

## Structured logger

All backend narrative writes go through `_log(kind, agent, text, extra)`. Timestamps use millisecond precision. Colour is ANSI-optional and disabled when the terminal doesn't look like a TTY or when the operator exports `NO_COLOR=1`. The header is on its own line so narrow terminals don't hide the timestamp when the message text wraps.

```

def _log(kind, agent, text="", extra=""):
    ts = datetime.now().strftime("%H:%M:%S.%f")[:-3]
    print(f"{ts} [{kind}] {agent} ({extra})")
    if text: print(f"    {' '.join(str(text).split())}")

```

## Endpoints registered on main.py

| Endpoint              | Method | Purpose                                                                                |
|-----------------------|--------|----------------------------------------------------------------------------------------|
| /                     | GET    | Health/root.                                                                           |
| /state                | GET    | Snapshot of <code>CityState.to_dict()</code> .                                         |
| /stream               | GET    | SSE broadcast (per-client queue, heartbeat every 1s).                                  |
| /event                | POST   | Low-level event injection (scripted replay).                                           |
| /invoke_scenario/{id} | POST   | Fire scripted scenario SC01/02/03.                                                     |
| /custom-incident      | POST   | JSON-only citizen submission; runs supervisor + graph.                                 |
| /report               | POST   | Multipart; accepts image, runs MobileViT, then delegates to <code>_run_custom</code> . |
| /route                | POST   | OSRM route between two points; supports pre-cached SC-01 corridor.                     |
| /geocode              | POST   | Nominatim wrapper with UA header + rate-limit sleep.                                   |
| /hospitals            | GET    | Cached read of <code>backend/data/hospitals.json</code> .                              |
| /alerts/{department}  | GET    | Filtered department alert list for standalone dept portals.                            |
| /medical-assessment   | POST   | EMS Mode 2 field-brief summariser + hospital re-selection.                             |
| /medical-assessments  | GET    | Read-back of all recorded medical assessments.                                         |
| /mode/{mode}          | POST   | Toggle Flow / Emergency / Protocol operating mode.                                     |

## 5.2 The Agent Layer

Every agent lives in `backend/agents/` as a standalone module and exposes exactly one function: `run(state) -> state`. Agents mutate the shared `CityState` — appending to `state.messages`, populating `state.department_alerts`, and setting domain-specific private keys like `state.__dict__['_incident']` for downstream agents to consume.

### How agents call the LLM

The common thin wrapper is **`agents/_llm.py`**. Its `ask(prompt, fallback)` function returns either the Groq completion text or the supplied deterministic fallback when the API is unavailable. This is what lets the whole system remain functional even when `GROQ_API_KEY` is missing or Groq is down — every LLM-facing agent supplies a fallback that produces a still-useful decision string.

```
# agents/_llm.py
def get_llm():
    key = os.environ.get("GROQ_API_KEY")
    if not key: return None
    from langchain_groq import ChatGroq
    return ChatGroq(model="llama-3.3-70b-versatile", api_key=key, temperature=0.3)

def ask(prompt: str, fallback: str) -> str:
    llm = get_llm()
    if llm is None: return fallback
    try:
        resp = llm.invoke(prompt)
        return getattr(resp, "content", str(resp)).strip()
    except Exception:
        return fallback
```

## Agent responsibilities and outputs

| Agent                     | LLM? | What it computes                                                           |
|---------------------------|------|----------------------------------------------------------------------------|
| IncidentAgent             | yes  | Severity label + type + affected lanes + spillover risk + ETA.             |
| SupervisingAgent          | yes  | Picks 1 of 5 protocol keys from an allow-list.                             |
| PredictionAgent           | no   | Junction cascade forecast at T+5, T+15, T+30 minutes.                      |
| ConstraintAgent           | yes  | Validates proposed intervention against 5 policy rules; RAG-boosted.       |
| InterventionAgent         | yes  | Ranks 3 severity-scaled response options; RAG-boosted.                     |
| EmissionAgent             | no   | CO <sub>2</sub> saved vs baseline based on severity multiplier.            |
| SafetyAgent               | no   | 0-100 safety score using nearby-POI dict.                                  |
| PoliceAgent               | yes  | units_required, deployment_instructions, diversion roads. RAG-boosted.     |
| EMSAgent (Mode 1)         | yes  | ambulances_required, ems_priority, required_medical_capability.            |
| EMSAgent (Mode 2)         | yes  | Summarises paramedic field assessment; forbidden from inventing diagnoses. |
| HospitalRoutingAgent      | no   | Capability-matched hospital pick using combined signals. RAG-boosted.      |
| EmergencyCorridorAgent    | yes  | Green-wave corridor plan + turn list. RAG-boosted.                         |
| ProtocolDispatchAgent     | no   | Templated department alerts based on severity_config.                      |
| CommsAgent (was Narrator) | yes  | Three-audience brief: operator, public, audit — three separate Groq calls. |

## Passing state between agents

Agents share information by writing under-scored keys onto CityState. IncidentAgent writes `_incident`; SupervisingAgent writes `_supervisor_plan`; PoliceAgent writes `_police_assignment`; EMSAgent writes `_ems_dispatch` or `_ems_assessment`. HospitalRoutingAgent then unions signals from those keys to compute the required medical capability set. This convention avoids a large shared dataclass while still keeping the flow inspectable — everything lives on the same object.

## 5.3 The SupervisingAgent

The SupervisingAgent is the routing brain. It runs immediately after the IncidentAgent has classified the incident and picks exactly one of five response sequences from a strict allow-list. The LLM output is validated: if the model returns anything that isn't a known key, the code falls back to a safe default (congestion) so downstream execution never crashes on a hallucination.

```
ALLOWED_AGENT_SEQUENCES = {  
  "congestion":      [Incident, Prediction, Constraint, Intervention, Comms],  
  "minor_accident":  [Incident, Prediction, Constraint, Intervention, Police, Comms],  
  "major_accident":  [Incident, Prediction, Constraint, Intervention, Police,  
                      EMS, HospitalRouting, EmergencyCorridor, Comms],  
  "fire_hazmat":     [Incident, Constraint, Police, EMS, HospitalRouting,  
                      EmergencyCorridor, Comms],  
  "medical_emergency": [Incident, EMS, HospitalRouting, EmergencyCorridor, Comms],  
}
```

### Escalation rule

After the plan is picked, if the incident severity turns out to be 4 or 5 but the supervisor still chose **congestion**, the orchestrator escalates the plan to **major\_accident** and pushes a second SupervisingAgent message noting the escalation. This makes the system defensive against the LLM under-reacting to structured severity information.

### Selected plan is broadcast

Once the graph completes, the plan is emitted to every SSE client as a SUPERVISOR\_PLAN event so the frontend AgentWorkflowPanel can grey out any agent that wasn't part of the sequence and show the selected protocol name in a header badge.

## 5.4 Utilities and Services

### `utils/city_state.py`

The **CityState** dataclass is the single source of truth for the entire runtime. It holds junction density, agent messages, department alerts, narrator outputs, current severity, emergency flag, selected hospital, and a snapshot of the active `custom_scenario` for late-joining dashboards. To prevent unbounded growth over the lifetime of a long-running server, **push\_message** and **push\_alert** cap their target lists at 200 entries and drop the oldest when full.

### `utils/severity_config.py`

Five entries — one per severity level — that fully describe the behavioural response envelope: which agents fire, which departments receive alerts, how far signal extensions can go, how many police units deploy, what CO $\blacksquare$  multiplier applies, and which intervention options are on the table. Every level-scaled number in the system is derived from this file.

### `utils/alert_dispatcher.py`

Templated multi-department alerts. Given an incident type, severity, location, and selected hospital, it returns a list of alert dicts each containing `{department, icon, message, timestamp}`. Departments are filtered by **SEVERITY\_CONFIG[severity].departments\_alerted**, so low-severity events emit fewer notifications than high-severity ones.

### `utils/causal_graph.py`

Static in-code representation of junction adjacencies with per-edge propagation delays in minutes. Used by PredictionAgent to project a spillover forecast without needing an LLM call.

### `utils/counterfactual.py`

Runs an alternate-universe simulation of the same incident under a 'no-intervention' regime and reports the CO $\blacksquare$  and vehicle-minute delta. Consumed by the operator dashboard's CounterfactualPanel.

## 5.5 The src/ subsystem: RAG, Routing, Vision

The backend/src/ tree hosts three self-contained subsystems that are separate from the agent modules so they can evolve independently. They are all imported lazily so their heavy dependencies (torch, transformers) don't inflate cold-start time.

### src/rag/retriever.py — Keyword-scored RAG

On first call it walks src/rag/knowledge/, splits each .txt file into paragraphs longer than 40 characters, and stores them in a module-level dict. **retrieve\_context(query\_terms, top\_n=3)** returns (context\_string, policy\_refs) — the top-scored paragraphs plus the filenames they came from. Score is simply the count of query terms found in the paragraph (case-insensitive). This is intentionally cheap: no embeddings, no vector store, no cold-start cost, and every match is explainable by the operator by opening the referenced .txt.

```
from src.rag.retriever import retrieve_context
ctx, refs = retrieve_context(["lane closure", "signal phase", "school zone"])
# ctx → three most relevant paragraphs joined with \n\n
# refs → e.g. ['traffic_policies.txt']
# ...then prepend ctx to the LLM prompt and store refs on the agent's message.
```

### src/routing/osrm\_router.py — OSRM + Nominatim client

Async wrappers around two free public APIs. **get\_route** hits router.project-osrm.org with overview=full&geometries=geojson&alternatives=true&steps=true. It converts OSRM's [lng, lat] coordinate pairs into Leaflet's [lat, lng] shape and parses the step-by-step maneuvers. **geocode** hits Nominatim with the required user-Agent header and enforces a 1.1-second sleep to stay under the fair-use rate limit. A **PRE\_CACHED\_ROUTES** dict holds the SC-01 Silk Board → Manipal HAL route so the scripted demo never depends on network availability.

### src/vision/analyzer.py — MobileViT image classifier

Loads apple/mobilevit-small via the Transformers pipeline at module import time (about 22 MB, cached after first run). **analyze\_image(image\_bytes)** runs classification at top\_k=20 with a lowered 0.08 confidence floor, then scores the frame using an expanded set of vehicle, person and road-hazard labels. It returns a JSON-like dict with vehicle\_count, injury\_indicators, hazards, verification\_status and a plain-English note stating that visual match is not proof.

```
hazards = [l for l in labels if any(h in l for h in ROAD_HAZARD_LABELS)]
score = vehicle_count*2 + (2 if person_detected else 0) + len(hazards)
verification_status = (
    "supported" if score >= 5 else
    "needs_verification" if score >= 3 else
    "insufficient_evidence")
```

## 6. Frontend Deep-Dive

The frontend is a single Vite-built React app served from port 5173. All screens share the same SSE stream but each has an independent EventSource so the operator dashboard and every department portal update in lock-step without needing to coordinate through browser storage or postMessage.

### Routing map

| Path            | Component           | Purpose                                    |
|-----------------|---------------------|--------------------------------------------|
| /               | App.jsx (Dashboard) | Operator control room.                     |
| /citizen        | CitizenView         | Public report + route planner.             |
| /dept/police    | PoliceDashboard     | Traffic Police portal.                     |
| /dept/hospital  | HospitalDashboard   | Hospital coordination + capability matrix. |
| /dept/fire      | FireDashboard       | Fire and hazmat portal.                    |
| /dept/public    | PublicDashboard     | Kiosk-style public info board.             |
| /dept/bbmp      | BbmpDashboard       | Audit trail + BBMP alerts.                 |
| /dept/ambulance | AmbulanceDashboard  | Paramedic field-input + live corridor.     |

### App.jsx flow

The Dashboard function opens exactly one EventSource to `http://localhost:8000/stream`. The `onmessage` handler dispatches on `evt.type` — `SET_DENSITY`, `ADD_MESSAGE`, `SET_EMERGENCY`, `SET_HOSPITAL`, `ADD_ALERT`, `SET_NARRATOR`, `INCREMENT_METRICS`, `SET_COMPLETION`, `SET_CUSTOM_BANNER`, `SET_SEVERITY`, `SUPERVISOR_PLAN`, `MEDICAL_ASSESSMENT`, `AGENT_STATE`, `SCENARIO_LOADED` — and updates the corresponding React state. A visible **SSE counter** in the top-right of the map card shows total events, message count, alert count, and the last agent name so mis-wiring is immediately obvious during a demo.

### SSE auto-reconnect

Native EventSource will silently retry after 3+ seconds when the connection is dropped. Because the backend can be restarted during development, App.jsx wraps the connection in a **connect()** closure that fires a 2-second deterministic reconnect on any `onerror`, closing the current socket first. The visible SSE indicator in the header flips between LIVE and DOWN so operators know when the backend hiccupped and when data resumed flowing.

### useTypewriter — cancellable message reveal

AgentTraceLog reveals each new message character-by-character. The custom hook maintains a monotonic `runRef` counter that **clear()** bumps. Any in-flight `setTimeout(step)` chain checks the `runRef` before running; if the caller cleared the log (e.g., a new scenario started), stale steps short-circuit instead of clobbering the new queue. Without this the old scripted-scenario reveals would race the new custom-incident reveals and drop lines.



## Components at a glance

| Component           | What it does                                                                             |
|---------------------|------------------------------------------------------------------------------------------|
| Header              | URBANMIND wordmark, SYSTEM ACTIVE dot, clock, mode buttons.                              |
| CockpitSidebar      | Scenario picker, active mode, system status, department portal links.                    |
| CityMap             | Leaflet map with Stadia dark tiles, junction density markers, incident ring, ambulance c |
| AgentWorkflowPanel  | Numbered step list of agents. Rows honour SupervisorPlan.sequence + severity.agent       |
| AgentTraceLog       | Typewriter-revealed message stream, coloured per agent.                                  |
| NarratorPanel       | Three stacked briefs: OPERATOR, PUBLIC, AUDIT.                                           |
| DepartmentAlertFeed | Slide-in alert cards, filtered by department in the portal views.                        |
| MetricsStrip        | Vehicle-minutes saved, CO■ avoided, corridors cleared, incidents prevented, all severi   |
| CausalGraphSVG      | Junction propagation graph, active edges animated cyan.                                  |
| TemporalMind        | T+5/15/30 prediction bars.                                                               |
| CounterfactualPanel | With / without UrbanMind side-by-side.                                                   |
| RoutePanel          | OSRM ETA/km summary + alternates.                                                        |
| CitizenView         | Report incident form, image upload, route planner, geolocation, 'open operator dashbo    |
| ErrorBoundary       | Wraps AmbulanceDashboard so any Leaflet edge-case renders a readable error instea        |

## 7. Data Flow (Citizen → Agents → Dashboards)

The end-to-end path a citizen incident takes through the system is the single most important flow to understand. Every other flow (scripted scenarios, medical assessment updates, department-only pages) is a variation on this template.

### Step 1 — Submission

The citizen fills the ReportingChat card on `/citizen`: description, severity 1–5, location string, optional photo. On submit the frontend POSTs either `/report` (multipart with the image) or `/custom-incident` (JSON) to the backend.

### Step 2 — Image analysis (optional)

If an image was attached, MobileViT runs synchronously in `/report` and its verdict is returned in the response body immediately. If the frame shows two or more vehicles plus a person, the backend auto-bumps the severity to at least 4 and marks `is_accident=True` before continuing.

### Step 3 — Scenario preparation

`_run_custom` geocodes the location via Nominatim (falling back to Silk Board's coordinates if the lookup fails), builds a synthetic CUSTOM scenario dict, stores it on `city_state.custom_scenario`, and enqueues a `SCENARIO_LOADED` event. Every connected dashboard reacts by centring its map on the incident and dropping a pin.

### Step 4 — Graph invocation

The agent graph runs on a worker thread via `asyncio.to_thread` to keep the event loop free. Inside the thread, `IncidentAgent` runs first (its output is what the supervisor needs to pick a plan). `SupervisingAgent` then chooses a protocol from the allow-list. The remaining agents fire in order; each RAG-retrieves policy context, then calls Groq via `ask()` and pushes its decision through `state.push_message`, which — thanks to the monkey-patch inside `_run_graph_streaming` — enqueues an `ADD_MESSAGE` plus an `AGENT_STATE` event on the SSE stream.

### Step 5 — Hospital selection

`HospitalRoutingAgent` combines the required-capability signals: field assessment specialty (if a paramedic has already reported) beats EMS dispatch requirement beats inferred capability from incident type. It filters the 30-hospital roster, ranks by distance from the incident origin, and emits a `SET_HOSPITAL` event. Every dashboard uses that to draw the destination marker and — via a downstream POST `/route` — the OSRM road corridor.

### Step 6 — Department fan-out

After the graph completes, `AlertDispatcher.dispatch` fires templated messages for every department in `SEVERITY_CONFIG[severity].departments_alerted`. The operator dashboard shows them in the `DepartmentAlertFeed`, and every `/dept/*` tab picks up the ones matching its filter substring — Traffic Police, Hospital, Fire, Public, BBMP.

## Step 7 — Metrics

Once alerts have been queued, the backend computes severity-scaled `vehicle_minutes_saved`, `co2_avoided_kg`, `corridors_cleared`, and `incidents_prevented`, then emits an `INCREMENT_METRICS` event. The MetricsStrip flashes green on any increment.

## Step 8 — Late-joiners

A dashboard opened *after* the incident is already in flight receives a `SNAPSHOT` event on its first SSE frame that contains `city_state.to_dict()`. If `custom_scenario` is present in the snapshot, the frontend hydrates the map, hospital, and severity from it — so the operator coming in late still sees the pin, the route, and the department alerts as long as the incident is still live (`custom_scenario` is cleared 20 seconds after submit).

## 8. API Reference

All endpoints accept and return JSON unless noted. CORS is wide open (allow\_origins=["\*"]) because the frontend runs on a different port during development.

### POST /custom-incident

```
Content-Type: application/json
{
  "description": "Truck overturned on Hosur Road",
  "severity": 4,
  "location": "Whitefield"
}
→ 200 { "ok": true, "queued": true }
```

### POST /report — multipart

```
Content-Type: multipart/form-data
fields: description, severity, location, image (optional file)
→ 200 { "ok": true, "image_analysis": { verification_status, confidence, ... } }
```

### POST /route

```
{
  "origin_lat": 12.9177, "origin_lon": 77.6228,
  "dest_lat": 12.9516, "dest_lon": 77.6648,
  "alternatives": true,
  "cache_key": "J1_to_manipal" // optional; bypasses OSRM
}
→ { "primary": { geometry:[[lat,lng],...], distance_km, eta_minutes, instructions:[...] },
    "alternatives": [ { ... }, { ... } ] }
```

### POST /geocode

```
{ "query": "Malleshpalya, Bangalore" }
→ { "lat": 12.97, "lng": 77.67, "display": "..." }
    or { "error": "Location not found – try being more specific..." }
```

### POST /medical-assessment

```
{ "incident_id": "custom",
  "raw_assessment_text": "HR: 120 bpm | BP: 90/60 | GCS: 12 | RTA, head laceration" }
→ { "ok": true,
    "summary": { condition_summary, consciousness_level, injury_summary,
                  recommended_preparations, specialty_needed },
    "hospital": { name, lat, lng, trauma, ... } }
```

## GET /stream — Server-Sent Events

Response: text/event-stream

```
data: {"type": "SNAPSHOT", "data": { ... }}\n\n
data: {"type": "HEARTBEAT", "ts": "..."}\n\n
data: {"type": "SCENARIO_LOADED", "data": {...}}\n\n
data: {"type": "SET_EMERGENCY", "data": {"value": true}}\n\n
data: {"type": "ADD_MESSAGE", "data": {"agent": "IncidentAgent", "text": "...", "color": "#f59e0b"}}\n\n
data: {"type": "AGENT_STATE", "data": {"agent": "IncidentAgent", "status": "completed", ...}}\n\n
data: {"type": "ADD_ALERT", "data": {"department": "Traffic Police", ...}}\n\n
data: {"type": "SET_HOSPITAL", "data": {"name": "St Johns Medical College", ...}}\n\n
data: {"type": "SUPERVISOR_PLAN", "data": {"sequence_key": "major_accident", ...}}\n\n
data: {"type": "INCREMENT_METRICS", "data": {"vehicle_minutes_saved": 1000, ...}}\n\n
```

## 9. LLM + RAG Pipeline

The reasoning surface of the system is small and identical across agents: build a keyword-scoped context, prepend it to a purpose-specific prompt, call `ask()`, and parse the response. The pattern is intentionally uniform so any agent's behaviour can be reasoned about in isolation, and so a Groq outage never crashes the response — the fallback string supplied to `ask()` is always a valid decision.

### The full LLM invocation for one agent

```
# agents/constraint.py – simplified
from ._llm import ask
from src.rag.retriever import retrieve_context

def run(state):
    ctx, refs = retrieve_context([
        "lane closure", "signal phase", "school zone",
        "bus stop", "monsoon", "speed"])

    prompt = (
        "You are the ConstraintAgent for an autonomous traffic system.\n"
        "POLICY CONTEXT:\n" + ctx + "\n\n"
        "INCIDENT: severity=" + str(state.severity) +
        ", type=" + str(state.__dict__.get('_incident', {}).get('type')) + "\n"
        "Respond in 1 short sentence: APPROVED or REJECTED, plus 1 reason.")

    fallback = "All active policies within safe operating envelope."
    text = ask(prompt, fallback)

    state.__dict__["_last_policy_refs"] = refs
    state.push_message(
        "ConstraintAgent",
        f"[{text}] [refs: {' '.join(refs)}]" if refs else text,
        "#ef4444")
    return state
```

### Why keyword RAG

Vector-search RAG systems (embeddings + a vector DB) are attractive but they add a heavy first-boot cost — sentence-transformers, torch, and chroma pull hundreds of megabytes and slow the cold path considerably. The observation that drove the design: the ~200 sentences in the SOP corpus can be scanned with a simple substring counter in under a millisecond, and the resulting matches are auditable because they are copied verbatim from a specific .txt. This trades recall for legibility and speed — a good bargain for a public-safety demo.

### Fallback discipline

Every LLM call has a fallback string chosen to be safe under all incident types. PoliceAgent's fallback dispatches units based on severity, EMSAgent's fallback picks a P1 priority, ConstraintAgent's fallback approves the intervention. When `GROQ_API_KEY` is unset — or when Groq is throttling — the system continues to produce a coherent chain of agent messages that the operator can still act on.

# 10. Networking: SSE, HTTP, CORS, Broadcast

## Why SSE and not WebSockets

The traffic flows in exactly one direction: the backend publishes to every dashboard, but the dashboards never push anything back over the same channel — commands (report incident, medical assessment, route lookup) are ordinary HTTP POSTs. Server-Sent Events give us that unidirectional stream over plain HTTP with built-in reconnect semantics in the browser and no framing library needed on the server. Compared with WebSockets, SSE is easier to log, easier to debug with `curl`, and easier to keep alive through basic reverse proxies.

## The broadcast fan-out

The single biggest bug fixed during development was the SSE stream using one shared `asyncio.Queue`. Each browser tab would drain items sequentially and starve the others. Replacing the shared queue with a per-client subscriber queue and a broadcast helper that iterates `_subscribers` guarantees fan-out. As a protection against subscriber leaks (aborted requests that never hit their finally block), the queue is capped at 20 and the oldest are dropped.

## Keep-alive with heartbeats

The SSE generator uses `asyncio.wait_for(q.get(), timeout=1.0)` so if no event has been queued for one second, the loop yields a HEARTBEAT instead. This does two things at once: it keeps corporate proxies from treating the connection as idle and closing it, and it lets the frontend detect a stalled backend by counting the gaps between heartbeats.

```
async def gen():
    yield f"data: {json.dumps({'type': 'SNAPSHOT', 'data': city_state.to_dict()})}\n\n"
    while True:
        try:
            evt = await asyncio.wait_for(q.get(), timeout=1.0)
            yield f"data: {json.dumps(evt)}\n\n"
        except asyncio.TimeoutError:
            yield f"data: {json.dumps({'type': 'HEARTBEAT', 'ts': datetime.utcnow().isoformat()})}\n\n"
```

## Buffering headers

Some browsers and proxies aggressively buffer text/event-stream responses, which makes agent messages arrive in bursts of 10-20. The stream endpoint sets `Cache-Control: no-cache, no-transform` and `X-Accel-Buffering: no` so intermediaries flush each SSE frame immediately.

## CORS

During development the frontend is on port 5173 and the backend on 8000. FastAPI is configured with `allow_origins=["*"]`, `allow_methods=["*"]`, and `allow_headers=["*"]`. If the two are ever served from the same origin (say, behind an nginx reverse proxy), the CORS middleware is a no-op and can be tightened.

## External API hygiene

- Nominatim requires a real User-Agent and 1 request per second per client. The wrapper enforces both.
- OSRM is best-effort; the code always accepts a route response with alternatives=false and returns a fallback straight-line if OSRM 5xxs.
- Groq calls are wrapped in try/except so an authentication failure or 429 never surfaces as an unhandled exception — the fallback path takes over.
- MobileViT is loaded lazily inside /report; opening the citizen page without submitting an image never triggers the download.



# 11. System Design Concepts

This section maps the design choices back to canonical system-design principles so the report can double as a study reference.

## Publish-Subscribe (fan-out)

The SSE broadcast is a textbook pub/sub with one publisher (the backend) and N subscribers (browser tabs). The subscriber list is decoupled from the publisher; adding or removing a subscriber requires no state on the publisher's call sites.

## Single Responsibility Principle

Each agent module exports exactly one `run(state)` function and depends only on the shared `CityState`. This makes the agents trivially unit-testable in isolation and lets the `SupervisingAgent` decide the sequence at runtime instead of hard-coding it in `graph.py`.

## Bulkheading

Failures in one agent don't propagate. Each dispatch inside `_Graph.invoke` is wrapped in `try/except` and reports a System error message on failure. Similarly, the `_never_500_middleware` guarantees a bad request handler doesn't kill the process.

## Backpressure / concurrency limits

A submit storm is throttled at the backend by an `asyncio.Semaphore(3)` around `_run_custom`. A fourth submit waits its turn rather than exhausting Groq quota or backing up MobileViT.

## Idempotency in stateful handlers

SSE reconnects re-deliver the SNAPSHOT. Every dashboard's snapshot handler is idempotent: state is only replaced when the incoming value is materially different. This is why the `AmbulanceDashboard` stopped blinking after we added guards on scenario id, hospital name, and alert length.

## Graceful degradation

The system's critical path never depends on external services. If Groq is down, agents fall back to deterministic stubs. If Nominatim fails, the incident falls back to Silk Board coords. If OSRM 5xxs, the route panel shows a synthetic straight line. If MobileViT can't load, image analysis is skipped and the incident is processed anyway.

## Observability

Every agent decision is emitted twice: once as an `ADD_MESSAGE` for the frontend, once as a structured log line to stdout. Every scenario invocation prints an `INTAKE / SCENARIO / GRAPH / PLAN` block bracketed by separator lines so a 60-second demo is trivially auditable in `tail -f`.

## Bounded resources

Two lists on CityState grow with every incident: **messages** and **department\_alerts**. Both are capped at 200 entries — appending beyond the cap drops the oldest. This is what prevented the ‘slow after N submits’ regression where the SNAPSHOT was ballooning past 500 KB and stalling the browser.

## Late-joiner reconciliation

The SNAPSHOT event carries the full CityState. Any dashboard opened mid-incident receives it as its first frame and hydrates itself. This means an operator can walk up to a running dashboard and see the exact same state as anyone else — no ‘reload to sync’ dance.

## 12. Logging Subsystem

Backend logging is deliberately handwritten rather than delegated to the standard library's logging module. The goal is a demo-grade terminal view: high-signal, colour-tagged, timestamped to the millisecond, and readable on any width of terminal.

### Format

```
HH:MM:SS.mmm [KIND] AgentName (extra=key=value)
  ■ full message text on its own line so wrap does not hide the header

# example
16:20:44.576 [MSG] IncidentAgent
  [HIGH · sev 4] INCIDENT_CONFIRMED at J1: severity 4/5, clearance ETA 30min. Spillover: J2, J5.
16:20:44.624 [MSG] SupervisingAgent
  Activating MAJOR_ACCIDENT protocol · agents: Incident,Prediction,... · priority HIGH
16:20:45.246 [GRAPH] orchestrator (took=755ms)
  invoke complete
```

### Colour handling

ANSI escape sequences are emitted only when stdout is a TTY, and never when the environment variable `NO_COLOR` is set. When colour is disabled the escape codes are empty strings, so the same log lines remain readable in a log file or a piped grep.

### Kinds and colour palette

| Kind        | When it fires                                         | Colour                           |
|-------------|-------------------------------------------------------|----------------------------------|
| INTAKE      | New citizen submission accepted.                      | red                              |
| SCENARIO    | Custom scenario built (after geocoding).              | red                              |
| GRAPH       | orchestrator invoke start / complete (with duration). | gray                             |
| MSG         | Any agent's push_message.                             | per-agent color                  |
| ALERT       | AlertDispatcher fires a department alert.             | gray                             |
| PLAN        | Supervisor plan finalised.                            | magenta                          |
| EVENT       | Scripted-scenario event replayed via POST /event.     | gray                             |
| MW-CATCH    | HTTP middleware caught an unhandled route exception.  | red (via stderr-like formatting) |
| TASK-CATCH  | asyncio.create_task raised without await.             | red                              |
| ASYNC-CATCH | Event-loop-wide unhandled exception.                  | red                              |

### Watching logs

The supervisor launches the backend in foreground when run interactively, so the log is already on your terminal. If the backend is nohup'd instead:

```
tail -f /tmp/urbanmind-backend.log
```

```
# scoped tail – just the agent messages
```

```
tail -f /tmp/urbanmind-backend.log | grep --line-buffered ' \[MSG\] '
```

```
# with colour re-enabled via less  
tail -F /tmp/urbanmind-backend.log | less -R
```

# 13. Deployment and Operations

## First-time setup

```
# clone
git clone https://github.com/haasa-x/urbanmind.git
cd urbanmind

# backend
cd backend
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
echo "GROQ_API_KEY=your_groq_key_here" > .env

# frontend
cd ../frontend
npm install
```

## Running (recommended, supervised)

```
./run.sh
# opens both, auto-restarts either on crash
# Ctrl-C stops both cleanly
# operator dashboard : http://localhost:5173/
# citizen page       : http://localhost:5173/citizen
# ambulance crew     : http://localhost:5173/dept/ambulance
```

## Running manually (two terminals)

```
# terminal 1 – backend
cd backend && source venv/bin/activate && uvicorn main:app --reload --port 8000

# terminal 2 – frontend
cd frontend && npm run dev
```

## Common operational commands

```
# nuke stale processes on both ports
lsof -ti:8000 -ti:5173 | xargs -r kill -9

# clear Vite dep cache (fixes stale bundles after multiple restarts)
rm -rf frontend/node_modules/.vite

# replay a scripted scenario against a running backend at 3x speed
cd backend && source venv/bin/activate
python replay.py scenarios/silk_board.json 3.0

# tail agent logs
tail -f /tmp/urbanmind-backend.log
```

## Ports and env

| Variable      | Default    | Purpose                                             |
|---------------|------------|-----------------------------------------------------|
| GROQ_API_KEY  | (required) | Groq API access. Read from backend/.env at startup. |
| NO_COLOR      | unset      | If set, backend log lines drop ANSI colour escapes. |
| Frontend port | 5173       | Vite dev server (--strictPort in run.sh).           |
| Backend port  | 8000       | uvicorn.                                            |

## 14. Future Work

- Replace the keyword RAG with a small local embedding index (sentence-transformers/all-MiniLM-L6-v2) once cold-start cost is measured against demo constraints.
- Persist CityState to Redis so the backend can be restarted without losing an in-flight incident.
- Add JWT + RBAC so operator, dispatcher, and citizen have differentiated endpoint access.
- Wire the OSMnx + NetworkX routing option so the ambulance corridor can account for live congestion weights rather than pure road-network shortest path.
- Auto-generate PDF incident reports (using reportlab, the same library that produced this document) with the full agent trace and CO<sub>2</sub> delta.
- Voice input on the citizen page for hands-free incident reports, backed by Whisper.
- Publish MobileViT weights inline via ONNX Runtime Web so image classification can happen client-side and the backend no longer requires torch.

# 15. Appendix: Command Reference

## Backend

```
# start
uvicorn main:app --port 8000 --reload

# start with logging colour off (for CI or non-TTY tails)
NO_COLOR=1 uvicorn main:app --port 8000

# fire a scripted scenario against a running backend
python replay.py scenarios/silk_board.json 3.0

# curl a custom incident
curl -s -X POST http://localhost:8000/custom-incident \
  -H "Content-Type: application/json" \
  -d '{"description":"Truck fire","severity":5,"location":"Hebbal"}'

# curl a medical assessment
curl -s -X POST http://localhost:8000/medical-assessment \
  -H "Content-Type: application/json" \
  -d '{"incident_id":"custom","raw_assessment_text":"..."}'

# tail the SSE stream to verify broadcast
curl -sN http://localhost:8000/stream
```

## Frontend

```
npm run dev          # start Vite dev server on :5173
npm run build        # production bundle to dist/
npm run preview      # serve the built bundle locally
```

## Ops

```
# kill all stale processes on the two ports
lsof -ti:8000 -ti:5173 | xargs -r kill -9

# check what's holding a port
lsof -i :8000

# check git identity that will author commits
git config user.name
git config user.email

# push to github
git push origin main
```

— END OF DOCUMENT —